

Digital Clock Project

Hi students,

Congratulations!

You are now venturing into the world of **interactive websites**, where user actions (like clicks) can dynamically change website content.

Before we jump into JavaScript and interactivity, we'll start by building a **static version** of the Digital Clock project.

Build a **static website** using **only HTML and CSS**.

At this stage:

-  No JavaScript logic
-  Hard-coded (static) time and date values

In the section where live time will eventually appear, **use static values** like:

- 12:31:07 PM
- 15/02/2026

HTML Structure

Use the following HTML structure:

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8" />

    <meta name="viewport" content="width=device-width,
initial-scale=1.0" />

    <title>Digital Clock</title>

    <link rel="stylesheet" href="styles.css" />

</head>
```

```
<body>

  <div class="container">

    <div class="clock-card">

      <h1>Digital Clock - Indian Standard Time</h1>

      <div id="time" class="time">21:08:30</div>

      <div id="date" class="date">05/02/2026</div>

      <button id="toggle-btn">Switch to 24-hour format</button>

    </div>

  </div>

  <script src="script.js"></script>

</body>

</html>
```

♦ **Note:**

Even though the `script.js` file is linked, you do **not** need to write any JavaScript yet.

CSS Styling

Use the following CSS to style your clock:

```
* {

  margin: 0;

  padding: 0;

  box-sizing: border-box;

  font-family: sans-serif;

}

body {
```

```
height: 100vh;

background: radial-gradient(circle at top, #1b245a, #050814);

display: flex;

justify-content: center;

align-items: center;
}

.container {

  text-align: center;
}

.clock-card {

  background: rgba(0, 0, 0, 0.75);

  padding: 40px 60px;

  border-radius: 14px;

  box-shadow: 0 0 40px rgba(0, 0, 0, 0.6);
}

.clock-card h1 {

  color: #ff4d6d;

  margin-bottom: 30px;
}

.time {

  font-size: 64px;

  color: white;

  margin-bottom: 15px;

  text-shadow: 0 0 12px rgba(255, 255, 255, 0.6);
}
```

```
.date {  
  
  color: #cfcfcf;  
  
  font-size: 18px;  
  
  margin-bottom: 30px;  
  
}  
  
button {  
  
  background: #ff4d6d;  
  
  color: white;  
  
  border: none;  
  
  padding: 12px 22px;  
  
  border-radius: 6px;  
  
  font-size: 15px;  
  
  cursor: pointer;  
  
  transition: background 0.3s ease;  
  
}  
  
button:hover {  
  
  background: #ff2e55;  
  
}
```

Expected Outcome

By the end of this step, you should have:

- A visually appealing digital clock UI
- Static time and date displayed

- A button styled and visible
- A project that is **ready to become interactive** in the next step 🚀

Once this is done, we'll move on to **JavaScript** and make the clock tick in real time.

Feature 1: Display Current Time on the Screen

Show the current time (hours, minutes, seconds) inside the clock UI and update it every second.

Steps:

1. Select the HTML element where time should be displayed using its id name `time` and store it in a variable called `timeElement`.
2. Create a function called `updateTime`.

3. Inside the function:

- Create a new `Date` object and store it in a variable called `now`.

Check about the date & time methods here: [JavaScript Date Reference](#)

- Extract:

- hours using `getHours()`
- minutes using `getMinutes()`
- seconds using `getSeconds()`

4. Convert minutes and seconds to always show two digits using `padStart(2, "0")`.

Explore more about

padstart:https://www.w3schools.com/jsref/jsref_string_padstart.asp

5. Display the time inside the `.time` element using `textContent`.

6. After setting the time using `textContent`, verify that it changes in the browser.

Steps:

1. Open the app in your browser

2. Open Console (press **F12**)

3. Type:

```
updateTime();
```

4. Press Enter

5. Observe the time on the screen

If the time updates instantly, your function is working correctly 

7. Use `setInterval()` to call the `updateTime` function every 1000 milliseconds.

Watch about `setInterval`

here: https://youtu.be/Ruq4sEw9h_8?si=rb6T3R1V0TZKj5uq

Feature 2: Display Current Date Below the Time

Show the current date in `DD/MM/YYYY` format.

Steps:

1. Select the HTML element with id `date`
Store it in a variable called `dateElement`.

2. Inside the `updateTime` function:

- Get the current date using:

- `getDate()`
- `getMonth()` (add +1 since months start from 0)
- `getFullYear()`

Check out how date methods work here: [JavaScript Get Date Methods](#)

3. Format the date properly and display it using `dateElement.textContent`.
-

Feature 3: Implement 12-Hour and 24-Hour Format Toggle

Allow the user to switch between 12-hour and 24-hour time formats.

Steps:

1. Create a variable called `is24Hour` and initialize it to `false`.
2. Select the toggle button using its ID `toggle-btn`
Store it in a variable called `toggleBtn`.
3. Inside the `updateTime` function:
 - If `is24Hour` is false:
 - Convert 24-hour time into 12-hour format.

Example: 18 → 6 PM, 15 → 3 PM, 00 → 12 AM
 - Decide whether to show `AM` or `PM`.
 - If `is24Hour` is true:
 - Display time as-is (24-hour format).
4. Append `AM` or `PM` only in 12-hour mode.

Feature 4: Toggle Button Functionality Using DOM Events

Change time format when the user clicks the button and update the button text.

Steps:

1. Add a click event listener to `toggleBtn`.
Read more about event listeners here: [JavaScript HTML DOM EventListener](#)
2. Inside the event listener:
 - Toggle the value of `is24Hour`.
 - If `is24Hour` is true then :
 - Change button text to **"Switch to 12-hour format"**
 - If it is false then :
 - Change button text to **"Switch to 24-hour format"**

Feature 5: Real-Time Clock Update

Ensure the clock updates automatically every second.

Steps:

1. Use `setInterval(updateTime, 1000)` to update the clock every second.
2. Call `updateTime()` once manually to avoid delay on page load.